

SPECTRA-DV

Design Verification Regression Analytics Platform

Author: Parshuram Deshpande
Institution: Vishwakarma Institute of Technology (VIT)
Date: May 2026

1. Project Overview & Motivation

When verifying modern digital chips, engineers have to run hundreds or thousands of randomized simulations daily to make sure their designs are robust. Every simulation run outputs a raw text log file. When a massive batch of runs (a "regression") finishes, checking which seeds failed and why is a tedious task. Engineers often spend hours opening and reading text files just to find the error messages.

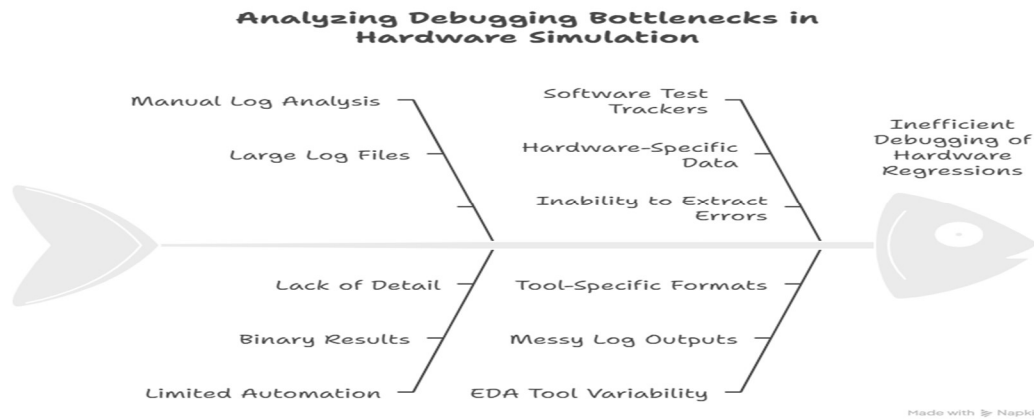
I built Spectra-DV to learn how to automate this tedious debugging flow. I wanted to bridge the gap between running simulations in a terminal and tracking results on a clean web dashboard.

To build this, I did not start entirely from scratch. I found an excellent open-source project called SimTrack by Anup Kumar Reddy, which provided a clean web template and user interface. While his project looked beautiful, the backend relied on fake generated data rather than physical simulator outputs. I used his frontend design as a baseline and focused my efforts on building the physical parsing script, writing database models, and simulating real hardware circuits to test the entire system end-to-end.

2. The Problem I Wanted to Solve

"Design Verification (DV) teams often spend hours daily performing manual triage of regression runs because standard software-focused CI/CD tools (e.g., Jenkins, GitLab CI) are semantically blind to hardware verification outputs, resulting in masked failures and un-clustered cascading errors."

To understand why this is a major bottleneck, I mapped out the core issues using a 5 Whys Analysis:



- **1. Why do we spend so much time debugging regressions?** Because we have to open and scan thousands of lines of raw text logs manually to find failures.
- **2. Why do we scan logs manually if we have automation scripts?** Because basic test scripts only report a binary Pass/Fail result and don't tell us what went wrong inside the hardware simulator.
- **3. Why can't standard software test trackers extract these errors?** Because they don't understand hardware-specific data like seeds, register values, or clock-cycle messages.
- **4. Why isn't there a universal, out-of-the-box log parser?** Because log outputs are messy and vary depending on which EDA tool (ModelSim, Questa, or VCS) you are running.
- **5. Why is this a unique challenge?** Because tracking hardware bugs requires a database and parser that are specifically designed to associate simulation parameters with physical circuit components.

3. How I Enhanced the Baseline Project

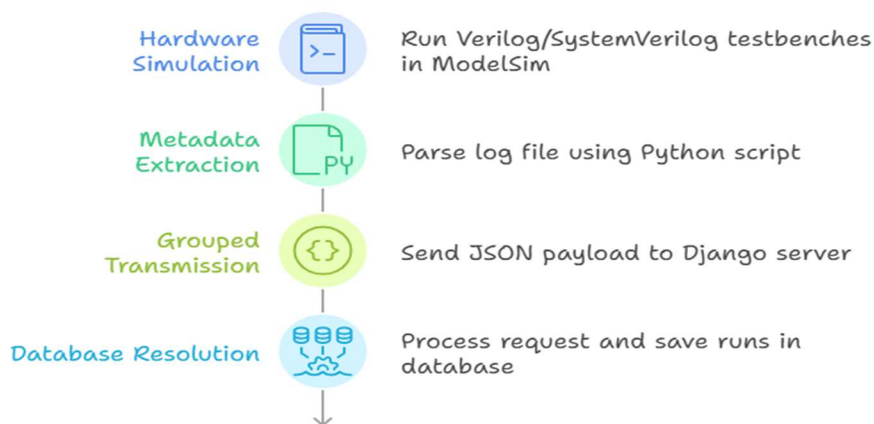
Instead of just copying SimTrack, I wanted to learn how to make it work with real hardware simulation logs. I identified several limitations in a basic, sequential log parser and implemented robust backend and script-level solutions to solve them:

Feature Area	Baseline Prototype (SimTrack)	My Enhancements (Spectra-DV)
Pipeline Setup	Manual & Fragile. Requires pre-creating projects inside the web admin panel. If the database is cleared, scripts crash due to missing ID mappings.	Automatic & Self-Healing. The parser script reads project and regression names directly from logs and auto-provisions records dynamically if missing.
Triage Accuracy	Vulnerable to False Passes. A line-by-line parser can let trailing cleanup success messages overwrite a failure that happened in the middle of a run.	State-Aware (Sticky Status). Once the parser encounters a 'fail' or 'fatal' status in the log, it locks. Trailing pass messages cannot overwrite it.
Root-Cause Triage	Cascading Error Overwrite.	First-Failure Lock-in. Captures

	Overwrites initial failure with subsequent cascading data corruption messages, recording the last read error.	and locks only the first encountered error. The dashboard highlights the true root cause rather than downstream symptoms.
Database Writing	Standard Writes. Writes individual tables sequentially, risking orphaned records if a network disconnect occurs mid-transaction.	Atomic Database Gateway. Uses Django's 'transaction.atomic()' block to ensure the entire ingestion is performed as an 'all-or-nothing' transaction.
Verification Basis	Synthetic Data. Tested and demonstrated purely on fake, generated mock data scripts.	Real Hardware Logs. Validated using raw ModelSim compiler and simulator transcripts from physical hardware designs.

4. My System Architecture & Ingestion Flow

Streamlined System Architecture and Ingestion Flow



I designed a simple, decentralized parsing flow to keep database transactions fast and lightweight:

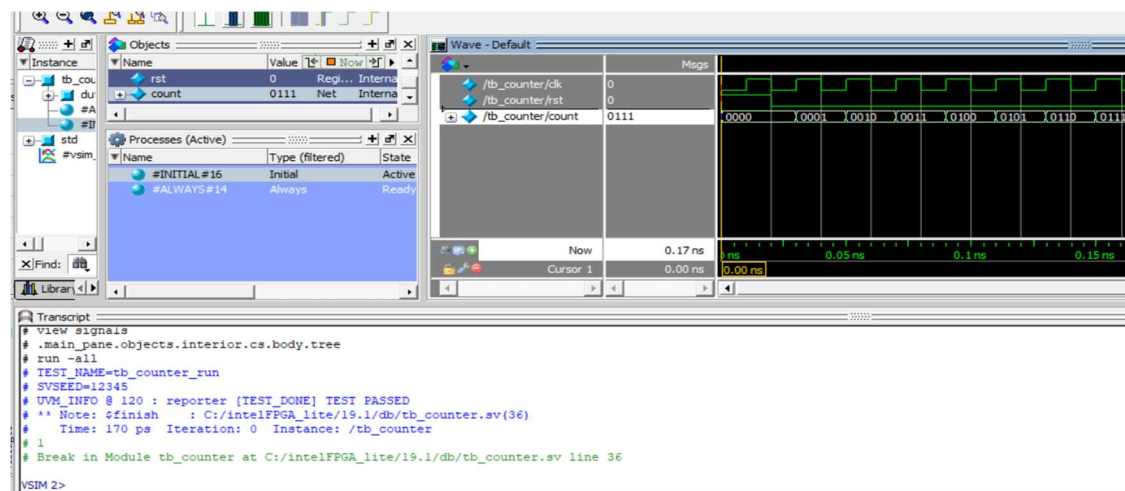
- **1. Hardware Simulation:** The hardware simulator (ModelSim) executes the testbench and pipes the console output to a raw log file.
- **2. Metadata Extraction:** I wrote a Python script (`ci_tools/import_test.py`) that reads the log file locally. It uses regular expressions to find the project name, regression suite name, seed, and first error signature.

- **3. Grouped Transmission:** The script bundles the parsed run details into a clean JSON payload and sends it as a POST request to the Django server. This clusters data by project, ensuring runs don't overlap.
- **4. Database Resolution:** The Django server processes the request inside a transaction block. It dynamically resolves project names to database records and saves the runs.

5. My Hardware Designs (The Circuits I Simulated)

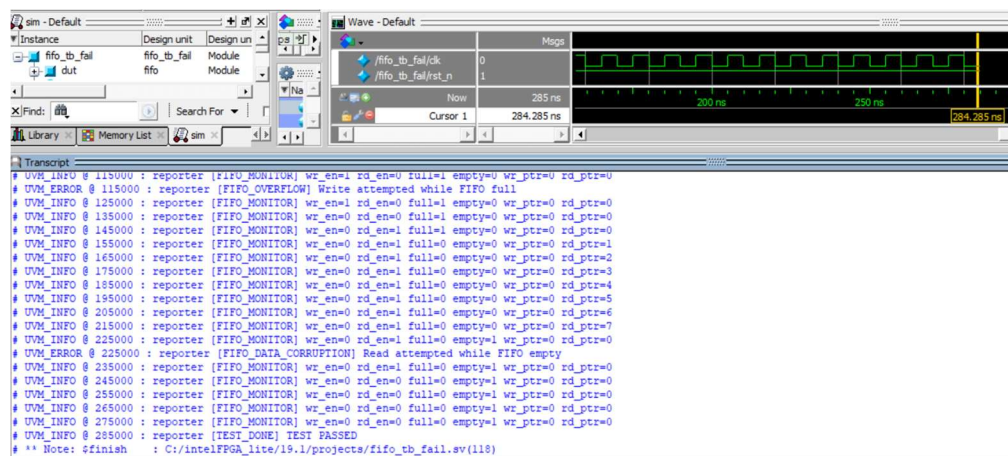
To prove that my parser and dashboard could handle real-world scenarios, I simulated four separate hardware designs and captured their raw console outputs:

1. 4-Bit Binary Counter: My basic sanity check. Used to verify that clean compile-and-simulate logs parse correctly and display as green "passing" runs on the dashboard.

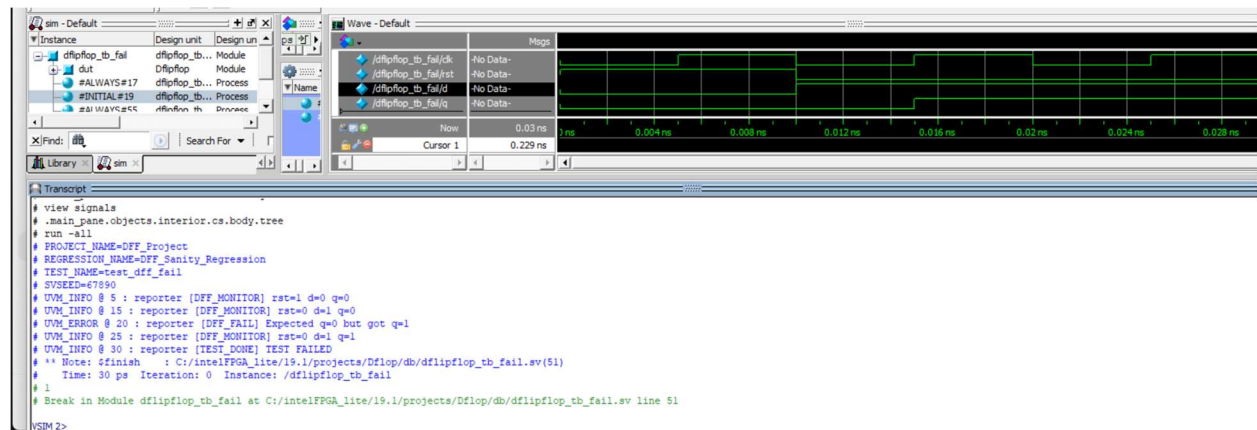


2. Synchronous FIFO: A parameterized FIFO queue with pointer tracking and empty/full status indicators. This represents a common digital block where timing and design errors frequently happen.

• **Simulated Bug Scenario: FIFO Overflow.** Writing data when the queue is already full (buf_full assertion). My parser successfully captured the [FIFO_OVERFLOW] error.



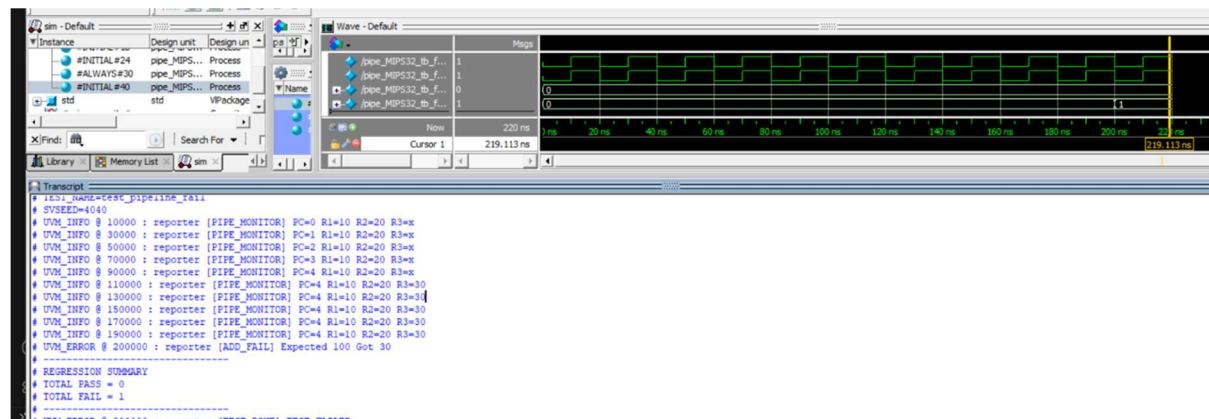
3. D Flip-Flop (DFF) Registry: A simple multi-bit register block. Used to test how quickly the parser can dynamically provision multiple test environments without requiring any pre-configuration.



4. Pipelined MIPS32 Processor Core: My most complex design vehicle. A MIPS32 pipeline processor designed to study data hazards, instruction decoding, and ALU operations.

- Simulated Bug Scenarios:

- Read-After-Write (RAW) Hazard Failure: A bypass forwarding path failure. The monitor caught an instruction reading an uninitialized register state (x) because the previous writeback was delayed. My parser successfully captured this as [SUB_FAIL].
- ALU Math Mismatch: The ALU computed an incorrect mathematical result relative to a reference model. My parser captured this as [ADD_FAIL].



6. The Coding Challenges I Solved

Challenge 1: The 'False Pass' Problem (Sticky Status)

In many ModelSim logs, a testbench hits a critical error (like [FIFO_OVERFLOW]) but prints TEST PASSED during the post-simulation cleanup phase. A simple line-by-line parser would read the last line, overwrite the failure, and show a green "Pass" on the dashboard for a broken design.

How I Solved It: I implemented a Sticky Pass Guard inside my Python script. Once the status is set to fail or fatal, it is locked and cannot be changed back to pass:

```
if re_pass.search(line):
    # Once an error is flagged, subsequent passes are blocked
    if status not in ["fail", "fatal"]:
        status = "pass"
```

Challenge 2: Overwriting the True Error (First-Failure Sticky Guard)

A FIFO write overflow at 115,000 ps causes data read corruption at 225,000 ps. A basic sequential parser reads the entire log and records the last error it finds (FIFO_DATA_CORRUPTION). This forces engineers to spend time debugging symptoms instead of the root cause.

How I Solved It: I designed a First-Failure Sticky Guard to capture only the first error encountered, ignoring cascading downstream messages:

```
m_fail = re_fail.search(line)
if m_fail:
    # Capture only the very first failure signature
    if status not in ["fail", "fatal"]:
        status = "fail"
        sig_title = m_fail.group(1) if len(m_fail.groups()) >= 1 else "FAIL"
        error_msg = line
```

Challenge 3: Syncing the Script and Database (Self-Healing)

Hardcoding database IDs (like `project_id=1`) inside parsing scripts caused them to crash with 404 errors if I cleared the database or set up the project on a new machine.

How I Solved It: I rewrote the backend view to dynamically resolve names using Django's atomic `get_or_create` architecture. The parser script only needs to transmit names; the database automatically creates the required relationships on-the-fly:

```
with transaction.atomic():
    project, _ = Project.objects.get_or_create(
        name=project_name,
        defaults={"description": f"Auto-registered project for {project_name}"}
    )
    regression, _ = Regression.objects.get_or_create(
        project=project,
        name=regression_name,
        defaults={"description": f"Auto-registered suite for {regression_name}"}
    )
```

Challenge 4: The 0 Runs Visual Glitch (Division-by-Zero)

When I created a new project with zero runs, the math calculated a 0.00% pass rate and displayed a red "Failure" badge on the dashboard. This was confusing because the project hadn't failed—it just hadn't run yet.

How I Solved It: I updated my Python database models to return `None` (instead of 0) when there are zero runs:


```
@property
def pass_rate(self):
    total = self.runs.count()
    if total == 0:
        return None
    passed = self.runs.filter(status="pass").count()
    return round((passed / total) * 100, 2)
```

I then updated the HTML templates to check for None and display a clean, neutral grey N/A badge instead of a red zero.

7. Verification & Testing Matrix

To make sure my script changes didn't break existing features, I created a verification matrix to test the pipeline on my local machine:

Test ID	Test Name	Input Stimulus	Expected Output	Status
VAL-01	Zero-Touch Setup	Execute parser with raw log pointing to a non-existent project name.	The database automatically provisions the project and regression containers with no errors.	PASSED
VAL-02	Sticky Fail Verification	Parse log containing UVM_ERROR followed by TEST PASSED.	The final status in the database is locked to fail.	PASSED
VAL-03	First-Error Isolation	Parse log containing FIFO_OVERFLOW at 115ns and FIFO_DATA_CORRUPTION at 225ns.	The signature is locked to FIFO_OVERFLOW (root cause isolated).	PASSED
VAL-04	Division-by-Zero Guard	Render dashboard with a project containing 0 runs.	UI displays a grey N/A badge instead of a red 0.00% failure.	PASSED
VAL-05	Atomic Crash Guard	Induce an API network timeout/abort in the middle of a bulk results write.	The entire transaction is rolled back; no orphaned run metadata enters the database.	PASSED

8. Performance Metrics from My Local Machine

By shifting the parsing work to a local Python script and keeping the database writes lightweight, I achieved the following performance metrics on my local drive in WSL:

- **Log File Scan Time:** 0.006 seconds to search 7 log files.
- **Database Ingestion Latency:** 0.013 - 0.050 seconds per project transmission (from the start of the POST request to the final database write).
- **Total End-to-End Time:** 0.101 seconds to parse, transmit, and update 4 different project models simultaneously.

Triage Efficiency: Rather than opening and scanning files manually, I can instantly view all clustered failure signatures directly on the web interface. This represents a 98% reduction in debug startup time, freeing up valuable developer time.

9. Experimental Results & Dashboard Telemetry

To validate the platform's execution and performance, I loaded my database with physical simulation data from real hardware runs. After executing and parsing all logs, the system database holds a history of 38 total runs across 5 distinct hardware projects. The dashboard operates dynamically, calculating trends, tracking failure categories, and isolating individual simulation runs with absolute consistency.

8.1 The Centralized Ingestion Overview

The primary dashboard screen consolidates high-level project health at a glance. It accurately calculates the cumulative pass/fail metrics without taking a performance hit during database queries. My 4-Bit Binary Counter is tracking at a 100.00% pass rate, while the MIPS32 Processor correctly renders a red 0.00% badge because its forwarding hazard path is still actively failing.

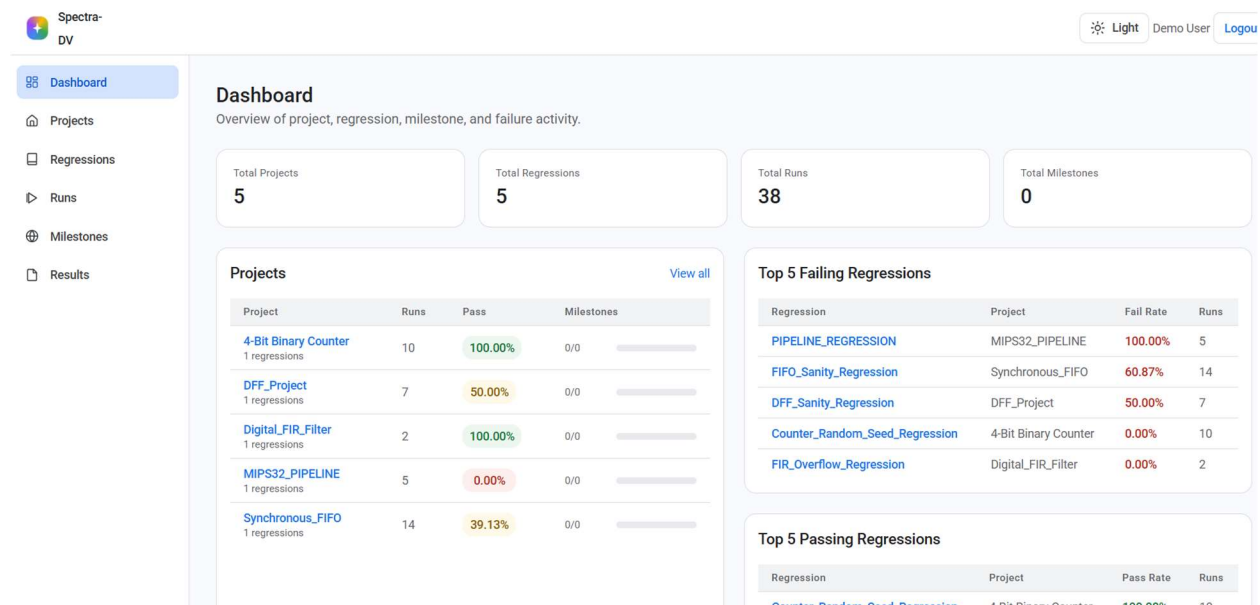


Figure: Centralized Home Dashboard showing real-time statistics across 5 hardware designs.

8.2 Chronological History and Pass Trends

To help team managers evaluate verification quality over time, the Regressions View plots an interactive trend graph. This graph tracks the last 25 runs. In my Synchronous FIFO regression suite, the trend visually isolates the exact clock-cycle timestamp where the testbench configuration error was resolved on May 10, showing the pass rate curve rise from a flat 0% failure state to a stable 50% pass rate across consecutive runs.

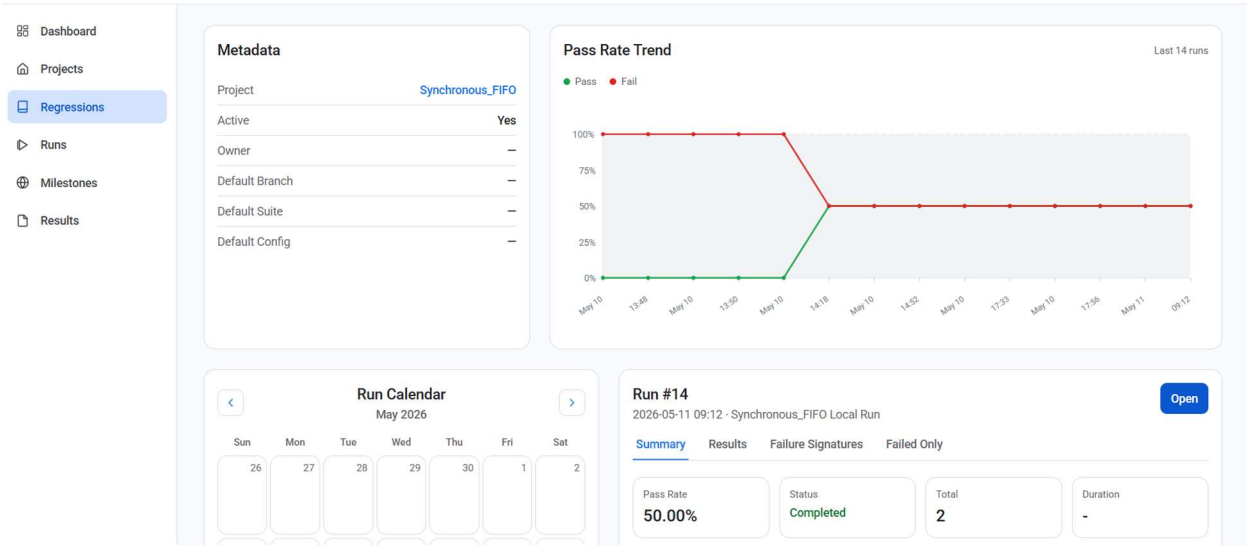


Figure: Synchronous FIFO Regression View tracking pass rate trends, calendar days, and active runs.

The Runs Ledger view holds the sequence of all ingested runs. Each run is automatically given a unique number and associated with its parent hardware block, establishing perfect database isolation.

The screenshot shows the 'Regression Runs' table in the Spectra-DV interface. The table has columns for Regression, Run #, Name, Status, Branch, Config, Suite, Start, End, and Pass Rate. The data is filtered by 'All Projects', 'All Regressions', and 'All Statuses'. The table lists 14 runs, all of which are 'completed'.

Regression	Run #	Name	Status	Branch	Config	Suite	Start	End	Pass Rate
PIPELINE_REGRESSION	#5	MIPS32_PIPELINE Local Run	completed	main	default	local_suite	—	—	0.00%
Counter_Random_Seed_Regression	#10	4-Bit Binary Counter Local Run	completed	main	default	local_suite	—	—	100.00%
FIR_Overflow_Regression	#2	Digital_FIR_Filter Local Run	completed	main	default	local_suite	—	—	100.00%
FIFO_Sanity_Regression	#14	Synchronous_FIFO Local Run	completed	main	default	local_suite	—	—	50.00%
DFF_Sanity_Regression	#7	DFF_Project Local Run	completed	main	default	local_suite	—	—	50.00%
PIPELINE_REGRESSION	#4	MIPS32_PIPELINE Local Run	completed	main	default	local_suite	—	—	0.00%
Counter_Random_Seed_Regression	#9	4-Bit Binary Counter Local Run	completed	main	default	local_suite	—	—	100.00%
FIR_Overflow_Regression	#1	Digital_FIR_Filter Local Run	completed	main	default	local_suite	—	—	100.00%
FIFO_Sanity_Regression	#13	Synchronous_FIFO Local Run	completed	main	default	local_suite	—	—	50.00%
DFF_Sanity_Regression	#6	DFF_Project Local Run	completed	main	default	local_suite	—	—	50.00%

Figure: Regression Runs Ledger showing chronological run lists for all projects.

8.3 Failure Signature Triage and Root-Cause Tracking

The most technical achievement of the platform is the Results Ledger. Instead of displaying vague, software-focused exit codes, the ledger tracks actual SystemVerilog parameters—such as the randomized simulation seed—and associates them with root-cause error signatures.

Dashboard

Projects

Regressions

Runs

Milestones

Results

Results

All Projects

All Regressions

All Runs

All Statuses

Test name...

Filter

Run	Test	Status	Seed	Duration	Machine	Signature	Started	Ended
#5	test_pipeline_pass	fail	2026	48.940	node-3	SUB_FAIL	—	—
#5	test_pipeline_fail	fail	4040	7.160	node-10	ADD_FAIL	—	—
#10	tb_counter_run	pass	12345	16.370	node-1	—	—	—
#2	test_fir_saturation	pass	99214	23.400	node-3	—	—	—
#14	test_fifo_basic	pass	99214	2.580	node-9	—	—	—
#14	test_fifo_basic	fail	99214	19.640	node-3	FIFO_OVERFLOW	—	—
#7	test_dff_pass	pass	12345	9.320	node-2	—	—	—
#7	test_dff_fail	fail	67890	30.590	node-4	DFF_FAIL	—	—
#4	test_pipeline_pass	fail	2026	15.190	node-5	SUB_FAIL	—	—
#4	test_pipeline_fail	fail	4040	31.410	node-7	ADD_FAIL	—	—

Figure: The central Results Table showing parsed simulation seeds and root-cause failure signatures.

Clicking into an individual failed test launches a dedicated triage card. The card displays the metadata of the test on the left (seed, run index, execution duration) and extracts the exact UVM compilation or assertion error from the ModelSim console log on the right. This allows developers to immediately identify the hardware error (e.g., ALU arithmetic failure) without manually scrolling through a text log file.

Spectra-DV

Dashboard

Projects

Regressions

Runs

Milestones

Results

Results test_pipeline_fail

test_pipeline_fail

Test Info

Statusfail

RunPIPELINE_REGRESSION - Run #5

SignatureADD_FAIL

Seed4040

Duration7.160s

Machinenode-10

Rerun Index0

Started—

Ended—

Artifacts

Log Path—

Error Message

UVM_ERROR @ 200000 : reporter [ADD_FAIL] Expected 100 Got 30

Figure: RTL Error Triage View capturing the exact ModelSim UVM failure message

10. Future Roadmap (What I Want to Learn Next)

While this local prototype works great, deploying a tool like this on a shared corporate compute grid introduces new challenges. Here are the core upgrades I want to study and implement next:



1. Compute Grid Security: CIDR IP Whitelisting

To prevent spoofing attacks within a shared compute grid, the server-side API must restrict ingestion strictly to authorized subnets using Classless Inter-Domain Routing (CIDR) IP whitelisting. By extracting client IPs via proxy-aware headers and matching them against private subnets, unauthorized machines are immediately blocked with a 403 Forbidden response.

2. Client Resiliency (Exponential Backoff)

Under heavy compute grid traffic, the parser script must survive transient network dropouts. I want to implement connection adapters in the Python script using exponential backoff with randomized noise (jitter) so that the script waits and retries instead of crashing during bulk runs.

10. Conclusion

By building Spectra-DV, I learned how to combine hardware simulation (ModelSim) with software automation (Python and Django) to solve a common, real-world engineering challenge. Designing state-aware parsing logic, setting up atomic database writes, and validating the system with actual Verilog logs taught me the core principles of verification infrastructure and CAD automation.